Materia: Laboratorio de Programación
EEST N.º 1 - “Eduardo Ader” Vicente López
5° año 3° Grupos A-B miércoles y jueves de 15:15 a 17:15 Hs



2° CUATRIMESTRE
🎮📺🛠️ ACTIVIDAD 7: MINI JUEGO "PIEDRA, PAPEL O TIJERA" CON PERSISTENCIA DE DATOS (struct + fstream) - LPR 5° 3° A-B (Presencial/Virtual Asincrónico - Actividades de Contingencia y Continuidad Pedagógica) 2026

🎯 OBJETIVOS DE LA ACTIVIDAD Y ALCANCE

Muchos programadores junior confunden la **Memoria RAM** (volátil: los datos se borran al cerrar el programa) con el **Almacenamiento Secundario / Disco** (no volátil: archivos de texto persistentes).

En la actividad 7 desarrollarán un **Mini Juego de Piedra, Papel o Tijera contra la Inteligencia Artificial de la Consola**, donde cada partida jugada se registrará automáticamente en una estructura (struct) y se guardará físicamente en un archivo de texto (historial_partidas.txt) en el disco rígido/estado sólido.

Objetivos Clave:

1. **Modelado con struct:** Agrupar variables heterogéneas (**ID**, **Jugador**, **EleccionJugador**, **EleccionIA**, **Resultado**) en una sola entidad.
2. **Escritura en Disco (ofstream):** Persistir los resultados en modo *Append* (ios::app) para no sobrescribir partidas previas del juego.
3. **Lectura desde Disco (ifstream):** Recuperar el historial completo almacenado en el archivo .txt y desplegarlo en la pantalla.

💡 CONEXIÓN CON EL PROYECTO INTEGRADOR ANUAL (para la EXPOTÉCNICA 2026)

- **Si su grupo tiene el P.I.A. avanzado:** Pueden adaptar los campos de la estructura Partida para registrar eventos reales de su prototipo (ej. lecturas de sensores, logs de usuarios, estado de stock, fichaje de turnos).
- **Si están demorados o trabajan de manera individual:** Desarrollen el juego de **Piedra, Papel o Tijera** tal cual está provisto en esta actividad. **Nadie debe quedarse sin entregar por Moodle.**

🧠 METODOLOGÍA DE APRENDIZAJE 5R

1. **R1 - REGISTRAR (Copiar y Compilar):** Copia el código fuente provisto en src/main.cpp, compilalo desde la terminal de PowerShell en VSCode y verifica que genere el archivo historial_partidas.txt.
2. **R2 - REDUCIR (Identificar el Núcleo):** Reconoce las líneas clave: ofstream archivo("historial_partidas.txt", ios::app); (escritura) e ifstream archivo("historial_partidas.txt"); (lectura).
3. **R3 - RESPONDER (Demostrar Razonamiento):** Responde el Cuestionario Técnico de Control explicando el funcionamiento del búfer de teclado y el flujo de archivos.
4. **R4 - REFLEXIONAR (Conectar con el P.I.A.):** Analiza cómo esta misma técnica permite guardar configuraciones o logs de hardware en un archivo .txt o .csv.
5. **R5 - REVISAR (Verificar y Documentar):** Realiza el git push a tus repositorios remotos (GitHub + GitLab) y adjunta el informe en PDF en Moodle.

📕 GLOSARIO TÉCNICO DE BOLSILLO

- **Memoria RAM (Volátil):** Espacio de trabajo temporal. Si el programa finaliza, los datos almacenados en variables se pierden.
- **Persistencia en Disco (No Volátil):** Técnica que escribe datos en el almacenamiento secundario (.txt, .dat, .csv) para conservarlos de forma permanente.
- **<fstream>:** Librería estándar de C++ especializada en el manejo de flujos de archivos (*File Streams*).
- **ofstream (Output File Stream):** Flujo de salida para **escribir/guardar** datos desde el programa hacia un archivo en el disco.
- **ifstream (Input File Stream):** Flujo de entrada para **leer/recuperar** datos desde un archivo en disco hacia el programa/juego.
- **Modo ios::app (Append):** Señal/Bandera de apertura que indica al sistema operativo que agregue nuevos datos al final del archivo sin borrar el contenido existente.
- **cin.ignore():** Función utilizada para limpiar el carácter *Enter* (\n) pendiente en el búfer de entrada antes de invocar getline().

🗂️ ESTRUCTURA OBLIGATORIA DEL REPOSITORIO EN GITHUB (piedrapapelotijera)

Tu espacio de trabajo local debe mantener la siguiente organización de archivos:

piedrapapelotijera/
├── .gitattributes <-- Normalización de finales de línea
├── .gitignore <-- Exclusiones (.exe, *.out, .vscode/)
├── LICENSE <-- Licencia MIT escolar
├── README.md <-- Portada del proyecto con integrantes
├── docs/ <-- Entregables y documentación
│ ├── InformeEEST1_LPR2026_ACT07_G03_Informe_v1.0.0.pdf <-- Informe APA v7
│ └── CHANGELOG.md <-- Historial de versiones del software
│ └── manuales
│ ├── manual_programador_v1.0.0.pdf
│ ├── manual_programador_v1.0.0.md
│ ├── manual_usuario_v1.0.0.pdf
│ ├── manual_usuario_v1.0.0.md
├── src/ <-- Código fuente compilable
│ ├── main.cpp <-- Programa principal del juego en C++
│ └── historial_partidas.txt <-- Archivo de persistencia generado
└── capturas/ <-- Evidencias de ejecución (.png)
├── jugada_consola.png <-- Captura del menú y partida jugada
└── archivo_disco.png <-- Captura del archivo .txt abierto

💻 CÓDIGO FUENTE DE REFERENCIA REPRODUCIBLE (src/main.cpp)

```
/*
  ========================================================================
  Materia: Laboratorio de Programación (LPR) — 5° Año
  Institución: E.E.S.T. N° 100 "Toy Story" — Vicente López
  Archivo: src/main.cpp
  Actividad 7: Mini Juego Piedra, Papel o Tijera + Persistencia en Disco
  ========================================================================
*/

#include <iostream>
#include <fstream>
#include <string>
#include <cstdlib>
#include <ctime>

using namespace std;

// 1. Estructura de datos para representar una partida
struct Partida {
    int idPartida;
    string jugador;
    string eleccionJugador;
    string eleccionIA;
    string resultado;
};

// Prototipos de funciones modulares
void jugarPartida(int& contadorID);
void guardarEnDisco(const Partida& p);
void leerHistorialDisco();

int main() {
    // Inicializar la semilla para los números aleatorios de la IA
    srand(time(0));

    int contadorID = 1;
    int opcion = 0;

    cout << "======================================================" << endl;
    cout << "  LPR 2026 - ACTIVIDAD 7: PIEDRA, PAPEL O TIJERA      " << endl;
    cout << "  Estudiante / Grupo: [Su Nombre y Apellido]          " << endl;
    cout << "======================================================" << endl;

    do {
        cout << "\n--- MENU PRINCIPAL DE JUEGO Y PERSISTENCIA ---" << endl;
        cout << "1. Jugar Partida (Piedra, Papel o Tijera)" << endl;
        cout << "2. Ver Historial de Partidas Guardadas en Disco (.txt)" << endl;
        cout << "3. Salir del Sistema" << endl;
        cout << "Ingrese una opcion (1-3): ";
        cin >> opcion;

        if (opcion == 1) {
            jugarPartida(contadorID);
        }
        else if (opcion == 2) {
            leerHistorialDisco();
        }
        else if (opcion == 3) {
            cout << "\n[SISTEMA] Saliendo del programa... Datos resguardados en disco." << endl;
        }
        else {
            cout << "\n[ERROR] Opcion invalida. Reintente." << endl;
```

```
        }

    } while (opcion != 3);

    return 0;
}

// Función para procesar la jugada contra la IA
void jugarPartida(int& contadorID) {
    Partida p;
    p.idPartida = contadorID++;

    cout << "\n--- NUEVA PARTIDA #" << p.idPartida << " ---" << endl;
    cout << "Ingrese su Nombre / Nickname: ";
    cin.ignore(); // Limpieza de búfer
    getline(cin, p.jugador);

    int opcJugador = 0;
    cout << "\nElija su jugada:" << endl;
    cout << "1. Piedra" << endl;
    cout << "2. Papel" << endl;
    cout << "3. Tijera" << endl;
    cout << "Opcion (1-3): ";
    cin >> opcJugador;

    // Asignar selección del usuario
    if (opcJugador == 1) p.eleccionJugador = "Piedra";
    else if (opcJugador == 2) p.eleccionJugador = "Papel";
    else p.eleccionJugador = "Tijera";

    // Elección aleatoria de la IA (1 a 3)
    int opcIA = (rand() % 3) + 1;
    if (opcIA == 1) p.eleccionIA = "Piedra";
    else if (opcIA == 2) p.eleccionIA = "Papel";
    else p.eleccionIA = "Tijera";

    // Evaluar reglas del juego
    cout << "\n[JUGADA] " << p.jugador << " eligio: " << p.eleccionJugador << endl;
    cout << "[JUGADA] La IA eligio: " << p.eleccionIA << endl;

    if (p.eleccionJugador == p.eleccionIA) {
        p.resultado = "Empate";
    }
    else if ((p.eleccionJugador == "Piedra" && p.eleccionIA == "Tijera") ||
             (p.eleccionJugador == "Papel" && p.eleccionIA == "Piedra") ||
             (p.eleccionJugador == "Tijera" && p.eleccionIA == "Papel")) {
        p.resultado = "Victoria";
    }
    else {
        p.resultado = "Derrota";
    }

    cout << "=> RESULTADO: " << p.resultado << "!" << endl;

    // Persistir el resultado inmediatamente en el archivo
    guardarEnDisco(p);
}

// Función para ESCRIBIR en el archivo de texto en disco
void guardarEnDisco(const Partida& p) {
    // ios::app permite agregar al final sin borrar partidas anteriores
    ofstream archivo("src/historial_partidas.txt", ios::app);

    if (!archivo.is_open()) {
```

```
        cerr << "[ERROR] No se pudo abrir el archivo para guardar la partida." << endl;
        return;
    }

    // Guardado de datos separados por comas (Formato CSV)
    archivo << p.idPartida << ","
            << p.jugador << ","
            << p.eleccionJugador << ","
            << p.eleccionIA << ","
            << p.resultado << endl;

    archivo.close();
    cout << "[DISCO] Partida resguardada con exito en 'historial_partidas.txt'." << endl;
}

// Función para LEER el historial persistido desde el archivo
void leerHistorialDisco() {
    ifstream archivo("src/historial_partidas.txt");

    if (!archivo.is_open()) {
        cout << "\n[AVISO] Aun no existen partidas guardadas en disco." << endl;
        return;
    }

    cout << "\n======================================================" << endl;
    cout << "  HISTORIAL DE PARTIDAS RECUPERADO DESDE DISCO       " << endl;
    cout << "======================================================" << endl;

    string idStr, jugador, elJug, elIA, resultado;
    int total = 0;

    // Lectura secuencial delimitada por comas
    while (getline(archivo, idStr, ',') &&
           getline(archivo, jugador, ',') &&
           getline(archivo, elJug, ',') &&
           getline(archivo, elIA, ',') &&
           getline(archivo, resultado)) {

        total++;
        cout << "Partida #" << idStr
             << " | Jugador: " << jugador
             << " | " << elJug << " vs " << elIA
             << " | Resultado: " << resultado << endl;
    }

    archivo.close();

    if (total == 0) {
        cout << "[AVISO] El archivo de historial esta vacio." << endl;
    } else {
        cout << "\n[SISTEMA] Total de registros leidos desde disco: " << total << endl;
    }
    cout << "======================================================" << endl;
}
```

🏃 COMANDOS DE POWERSHELL DESDE VS CODE

Para compilar y ejecutar tu programa en Windows 10/11:

```
# 1. Compilación limpia habilitando C++11
g++ -std=c++11 src/main.cpp -o src/piedrapapelotijera.exe

# 2. Ejecución directa
.\src\piedrapapelotijera.exe
```

📋 CUESTIONARIO TÉCNICO DE CONTROL (OBLIGATORIO EN EL INFORME)

Respondan de forma clara las siguientes 5 preguntas en su informe APA v7:

1. **RAM vs. Disco:** ¿Por qué si cerras la ventana de consola y volves a abrir el ejecutable, la Opción 2 sigue mostrando el historial de partidas anteriores?
2. **Uso de ios::app:** ¿Qué sucedería con el archivo historial_partidas.txt si en la función guardarEnDisco utilizaras ofstream archivo("historial_partidas.txt"); sin incluir la opción ios::app?
3. **Manejo del Búfer (cin.ignore()):** ¿Por qué es necesario invocar cin.ignore() antes de leer el nombre del jugador con getline(cin, p.jugador)?
4. **Agrupación con struct:** ¿Qué ventajas ofrece la estructura Partida para organizar las variables del juego frente a pasar 5 variables sueltas a cada función?
5. **Aplicación al P.I.A.:** Si adaptaran este módulo de almacenamiento a su Proyecto Integrador Anual, ¿qué datos o eventos de su prototipo guardarían en el archivo de texto?

🛡️ 10. ESCUDO ANTI-EXCUSAS (RESOLUCIÓN EN AULA)

Duda típica del estudiante	Respuesta y Solución Técnica Inmediata
"No sé qué datos guardar."	**Solución:** Desarrolla el juego de Piedra, Papel o Tijera exactamente como está en el código base de la actividad 7.
"No me crea el archivo historial_partidas.txt."	**Solución:** El archivo se crea **automáticamente** la primera vez que se juega una partida (Opción 1). Revisar la carpeta src/.
"Se me saltea la pregunta del Nombre del Jugador."	**Solución:** Te falta colocar cin.ignore() antes de getline(). Verifica la línea 75 de la fuente en C++.
"En la PC de la escuela me da error de permisos para compilar."	**Solución:** Utiliza g++ **-std=c++11**(Comando estándar de compatibilidad C++11) u OnlineGDB (evidencia), peguen el código C++, prueben la ejecución, descarga los archivos y subirlos a tu repositorio de GitHub.

🏆 PAUTA DE ENTREGA Y EVALUACIÓN

1. **Formato de Entrega del informe APA v7 en PDF:** Informe en PDF redactado bajo normas APA v7 guardado en docs/InformeEEST1_LPR2026_ACT07_G03_[ApellidoNombre]_Informe_v1.0.0.pdf. con **versionado obligatorio**.
2. **Demostración de Persistencia:** Deben incluir capturas de pantalla de la terminal jugando una partida, cerrando la consola, volviéndola a abrir y mostrando el historial persistido de la Opción 2.
3. **El repositorio:** Sin importar si compilaste en tu computadora o en OnlineGDB, el repositorio propio de GitHub con toda la estructura de carpetas (src/, docs/, etc.) y el código subido con su modificación debe estar creado obligatoriamente. No se aceptan carpetas sueltas.
4. **Sincronización Git Dual-Remote:** Subir el código fuente y las capturas ejecutando la secuencia Dual-Push a GitHub y GitLab con el versionado indicado en la guía.
5. **Plataforma Moodle:** Adjuntar la URL pública de los repositorios en el informe APA V7 en PDF antes del plazo fijado.

🔗 Enlaces Técnicos de Referencia (Links de Consulta):

- 🔗 GeeksforGeeks — Structs y Estructuras de Datos en C++
- 🔗 Microsoft Learn — Flujos de Archivos en C++ (fstream, ofstream, ifstream)
- 🔗 cppreference — Modos de Apertura de Archivos (ios::app, ios::in, ios::out)
- 🔗 GeeksforGeeks — Manejo de Búfer de Entrada (cin.ignore()
- 🔗 Git Documentation — Git Remote & Dual Push Configurations

📅 Fecha Límite de Entrega: 16 de Octubre de 2026.